certora

Security Assessment

Final Report

Light Protocol Extension

February 2026

Prepared for Light Protocol

Table of Contents

Project Summary 4
- Project Scope 4
- Project Overview 4
- Protocol Overview 4

Assessment Methodology 5

Threat and Security Overview 6
- Findings Summary 8
- Severity Matrix 8

Detailed Findings 9
- High Severity Issues 12
  - H-01: Restricted extension checks can be bypassed by using only CToken compressions, without compressed outputs 12
  - H-02: User can bypass decompression validation by specifying a high compression index, escaping frozen state and stealing funds from owner 14
  - H-03: Decompression doesn't compare amounts for most cases, allowing escaping frozen state and stealing by delegate 15
  - H-04: Delegate can freeze funds by decompressing a frozen account into a regular account 17
- Medium Severity Issues 19
  - M-01: Compressed ATA can't decompress if the reopened ATA is frozen 19
  - M-02: Delegate cannot compress tokens from CToken account despite having delegation rights 20
  - M-03: Mint might be re-opened with different extensions, leaving the existing token accounts with wrong extension list 21
- Low Severity Issues 23
  - L-01: Top up would be charged twice from the same account for a self-transfer 23
  - L-02: Tokens decompressed to a cToken account pending closure are burned without decreasing cmint supply 24
  - L-03: Decompressing a compressed-only account might increase the delegation amount by more than it should 26
  - L-04: Permanent delegate can decompress a compressed-only account only into an account owned by the original owner 27
  - L-05: Frozen CToken account can't be closed, unlike SPL token specs 28

L-06: Compression authority can add CompressedOnly extension to compressed token accounts that don't need it.... 29
L-07: `max_top_up` max value isn't sufficient to support many use cases.... 30
L-08: Compressed only delegation should apply the delegated amount if delegation address is the same.... 31
L-09: Permanent delegate can't burn.... 32
L-10: During mint decompression we shouldn't allow `rent_payment` to be 0.... 33
L-11: An attacker can create an account before mint creation to avoid restricted extension detection.... 34
Informational Severity Issues.... 35
I-01: Compressed token accounts with zero amounts can be created for restricted mints.... 35
I-02: New rent exempt rate is covered by the fee payer but goes to the rent sponsor when closed.... 36
Disclaimer.... 37
About Certora.... 37

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Light Protocol Extension	https://github.com/Lightprotocol/light-protocol	13c64a1..bced0c7	Solana

Project Overview

This document describes the security review of **Light Protocol Extension**. The work was undertaken from **January 12th, 2026** to **February 13th, 2026**.

The following contract list is included in our scope:

program-libs/*
programs/*

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The Compressed Token Program is a Solana program that enables rent-free, compressed token accounts for efficient token storage and distribution, supporting both SPL Token and Token-2022 mints. It uses Merkle trees for compressed account storage and provides SPL-compatible interfaces while adding compression capabilities.

Additionally, the program allows funds to be stored in standard Solana accounts, with the option to sponsor rent exemption in exchange for a fee.

This audit focuses on the project's latest updates, specifically the support of new extensions and the introduction of additional instructions for CTokens.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

While auditing the codebase, we've investigated various threats and attack vectors, among them:

Loss of funds for the protocol or a specific user:

- Transferring funds without the authorization of the owner
  - Attacker impersonating the permanent delegate
  - Bypassing the signature checks
  - Any other logical issue that breaks the checks
- Delegate having control over more than the delegated amount
- Unauthorized minting or creation of new tokens
- Cases of loss of funds without profit for the attacker (e.g. frozen funds)
- Rent sponsor funds can be stolen
  - e.g. if we can open an account with the rent sponsor but then not return the rent-exempt when the account is closed. This would allow draining the rent sponsor account.

Users breaking token restrictions:

- Moving frozen funds into a non-frozen account
- Creating compressed accounts for restricted mints without the 'compressed only' extension
  - This would allow bypassing the restrictions of the restricted extensions
- Transferring funds while paused
- Preventing the permanent delegate from transferring or burning from a token account
- Creating a token account with a state different from the default state of that specific mint
- Transferring a token with a transfer hook without executing the hook

Non-critical threats:

- Temporary DoS
- Delegate not being preserved while compressing/decompressing

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	4	4	3
Medium	3	3	2
Low	11	11	7
Informational	2	2	-
Total	20	20	12

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
H-01	Restricted extension checks can be bypassed by using only CToken compressions, without compressed outputs	High	Fixed
H-02	User can bypass decompression validation by specifying a high compression index, escaping frozen state and stealing funds from owner	High	Fixed
H-03	Decompression doesn't compare amounts for most cases, allowing escaping frozen state and stealing by delegate	High	Fixed
H-04	Delegate can freeze funds by decompressing a frozen account into a regular account	High	Acknowledged
M-01	Compressed ATA can't decompress if the reopened ATA is frozen	Medium	Acknowledged
M-02	Delegate cannot compress tokens from CToken account despite having delegation rights	Medium	Fixed
M-03	Mint might be re-opened with different extensions, leaving the existing token accounts with wrong extension list	Medium	Partially Fixed
L-01	Top up would be charged twice from the same account for a self-transfer	Low	Fixed

L-02	Tokens decompressed to a cToken account pending closure are burned without decreasing cmint supply	Low	Fixed
L-03	Decompressing a compressed-only account might increase the delegation amount by more than it should	Low	Acknowledged
L-04	Permanent delegate can decompress a compressed-only account only into an account owned by the original owner	Low	Acknowledged
L-05	Frozen CToken account can't be closed, unlike SPL token specs	Low	Fixed
L-06	Compression authority can add CompressedOnly extension to compressed token accounts that don't need it	Low	Fixed
L-07	`max_top_up` max value isn't sufficient to support many use cases	Low	Fixed
L-08	Compressed only delegation should apply the delegated amount if delegation address is the same	Low	Fixed
L-09	Permanent delegate can't burn	Low	Acknowledged
L-10	During mint decompression we shouldn't allow `rent_payment` to be 0	Low	Fixed

L-11	An attacker can create an account before mint creation to avoid restricted extension detection	Low	Acknowledged
I-01	Compressed token accounts with zero amounts can be created for restricted mints	Informational	Acknowledged
I-02	New rent exempt rate is covered by the fee payer but goes to the rent sponsor when closed	Informational	Acknowledged

High Severity Issues

H-01: Restricted extension checks can be bypassed by using only CToken compressions, without compressed outputs

Severity: High	Impact: High	Likelihood: Medium
Files:	Status: Fixed	

Description:

In build_mint_extension_cache(), if there are no output compressed tokens in transfer2(), restricted extension checks are skipped. The function uses parse_mint_extensions() instead of check_mint_extensions() when no_compressed_outputs is true, which does not enforce pause, transfer fee, or transfer hook validations.

An attacker can construct a transfer2() call that bypasses extension checks by setting:

- `compressions = [Compress from cToken_A, Decompress to cToken_B]`
- `in_token_data = []` (no compressed inputs)
- `out_token_data = []` (no compressed outputs)

Since out_token_data is empty, no_compressed_outputs evaluates to true, causing the bypass path to be taken for all compressions in the loop.

This leads to:

- **Pause bypass**: Transfer cTokens even when mint is paused
- **Transfer fee bypass**: Transfer without paying non-zero transfer fees
- **Transfer hook bypass**: Transfer without triggering transfer hook

For transfer fee bypass, when the mint has non-zero transfer fees:

- SPL → cToken via Compress then Decompress: The compressed balance is set to the full transfer amount, but the pool only receives the amount minus fees. This creates an accounting mismatch where there are insufficient funds to back all compressed balances.

- cToken → cToken transfers completely bypass transfer fees since no actual SPL transfer occurs.

Recommendations:

Enforce check_mint_extensions() whenever there's a Compress mode compression, regardless of whether compressed outputs exist

Customer Response:

Fixed in PR #2246

Fix Review:

An attacker can still specify SPL compression and decompression while transfer fee is active. The fee for the compression would be at the expense of the protocol.

Fixed in commit c7866ee, now also Token 2022 compressions are checked.

H-02: User can bypass decompression validation by specifying a high compression index, escaping frozen state and stealing funds from owner

Severity: High	Impact: High	Likelihood: Medium
Files: programs/compressed-token/program/src/compressed_token/transfer2/token_inputs.rs	Status: Fixed	

Description:

When decompressing a compressed-only account the caller must specify the compression_indexof the compression that contains the matching decompression.The program populates the compression_to_input array when iterating over the inputs, and then uses it to validate the match when iterating over the compressions.

However, we only check that the index is less than MAX_COMPRESSIONS. The caller can still specify a value that's higher or equal to the compressions length. In that case, we'll never check the input account ot have a matching compression.

This would allow the caller to decompress the compressed-only account without any restrictions. Moving frozen funds into a non-frozen account, or a different ownership (delegate signer is enough, even though the signer might have only a small part of the funds in the account delegated to them).

Recommendations:

Verify that compression_index is less than the compressions length.

Customer Response:

Fixed in PR #2233

Fix Review:

Looks good

H-03: Decompression doesn't compare amounts for most cases, allowing escaping frozen state and stealing by delegate

Severity: High	Impact: High	Likelihood: Medium
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/decompress.rs	Status: Fixed	

Description:

When decompressing a compressed only account we compare the amount only in case that the compressed input is an ata or a 'compress to pubkey' account, otherwise this check is skipped.

This would allow compressing the rest of the funds into a different account, bypassing all the checks of the 'compressed only' extension. Moving funds from a frozen input to a non-frozen account, or the delegator transferring the funds to an account with a different ownership (even though the delegator might be delegated only part of the funds, as low as 0).

Consider the following scenario:

- Bob has an account with 10K USDC
- Bob delegates 1 USDC to Eve
- The account gets compressed and closed
- Eve now executes `transfer2` to decompress the account
  - Eve provides 1 ctoken account owned by Bob to bypass the 'compressed only' check, but transfers 0 tokens into it
  - Eve provides a second CToken account to transfer the rest of the 10K USDC into an account she owns.
  - Eve has effectively stolen 10K USDC from Bob

Recommendations:

Check the decompressed amount even when it's not an ata or a 'compress to pubkey' account

Customer Response:

Fixed in PR #2235

Fix Review:

Looks good

H-04: Delegate can freeze funds by decompressing a frozen account into a regular account

Severity: High	Impact: High	Likelihood: Medium
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/decompress.rs	Status: Acknowledged	

Description:

During decompressing a compressed-only account, we freeze the destination account if the input account is frozen as well.

Considering that the delegate and permanent delegate also have the power to decompress an account, they would be able to freeze any CToken account (with the same mint) owned by the owner.

Exploit scenario:

- Bob has account A with 10 USDC
- Bob delegates 1 USDC to Eve
- Account A gets frozen
- Account A gets compressed and closed
- Bob opens a new account B and deposits there 10K USDC
- Eve decompresses account A into account B
- The full 10K USDC in account B is now frozen

Recommendations:

If the compressed account is frozen - check that the CToken account is forzen, or it has a 0 amount.

Customer Response:

Acknowledged - frozen state applies to the whole account

Medium Severity Issues

M-01: Compressed ATA can't decompress if the reopened ATA is frozen

Severity: Medium	Impact: High	Likelihood: Low
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/decompress.rs	Status: Acknowledged	

Description:

Compressed ATA accounts can get decompressed only into the same address they were compressed from.

In case the original ATA account got frozen the owner can't decompress the account, because the program rejects compressions for frozen accounts.

Recommendations:

Consider making an exception for such case and allow decompressing into another account.

Customer Response:

Acknowledged

M-02: Delegate cannot compress tokens from CToken account despite having delegation rights

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: programs/compressed-token/program/src/shared/owner_validation.rs	Status: Fixed	

Description:

The check_ctoken_owner() function validates authorization for compression operations on cToken accounts. Currently, it only permits the owner or the permanent delegate (from the mint) to compress tokens, but does not check for the account-level delegate.

In SPL Token, when an owner approves a delegate, that delegate gains the right to transfer tokens on behalf of the owner up to the delegated amount. Compression is functionally equivalent to transferring tokens out of a cToken account (moving them from on-chain state to compressed state). Therefore, a delegate should also be permitted to compress tokens from the cToken account they have delegation rights over.

Recommendations:

- Add a check for the account-level delegate in `check_ctoken_owner()` to allow delegates to compress tokens, consistent with SPL Token's delegate transfer behavior.
- When a delegate compresses tokens, decrease the `delegated_amount` by the compressed amount, mirroring how SPL Token decrements `delegated_amount` when a delegate transfers tokens.

Customer Response:

Fixed in PR #2262

Fix Review:

Looks good

M-03: Mint might be re-opened with different extensions, leaving the existing token accounts with wrong extension list

Severity: Medium	Impact: Medium	Likelihood: Low
Files: programs/compressed-token/program/src/ctoken/create.rs	Status: Partially Fixed	

Description:

Mints with the MintCloseAuthority can be closed and re-opened with a different extension list.In the meantime, all token accounts that were created with the old extension would have an extension list that matches the old mint.

Specifically, this can cause issues in the following scenarios:

1. In a case where the mint wasn't restricted beforehand, the token pools would be without the restricted seed.
   This would allow using the anchor instructions to compress SPL tokens into a compressed account, and then transfer it using Anchor's transfer ix.

2. Old accounts without the restrictions can be compressed and closed without the 'compressed-only' extension because the `compression_only` flag is set to false.

3. In case the previous mint had a transfer fee, and the new one doesn't.
   Then a CToken account with a fee config extension gets compressed and closed, now when the owner opens a new account to decompress it'd fail since it doesn't have the fee config

Recommendations:

1. Consider making the `MintCloseAuthority` a restricted extension

2. In the scenario of decompressing an account with fee config extension into an account without - allow doing so as long as the mint no longer has the fee config extension.

Customer Response:

Fixed in PR #2263

Fix Review:

This addresses points #1 and #2, the issue in point #3 still might happen.

Low Severity Issues

L-01: Top up would be charged twice from the same account for a self-transfer

Severity: Low	Impact: Low	Likelihood: Low
Files: programs/compressed-token/program/src/ctoken/transfer/shared.rs	Status: Fixed	

Description:

In case of a self-transfer via the CtokenTransfer and CTokenTransferChecked ixs, we'd be charging the top up lamports from the same account twice - once as the source account and another time as the destination account.It should be charged only once.

Recommendations:

In case of a self transfer charge the top up amount only once.

Customer Response:

Fixed in PR #2252, we now early return on self transfer and don't charge any top up

Fix Review:

Note that on SPL we revert if the account is frozen or the balance is less than the transfer amount.Besides that the fix looks good.

L-02: Tokens decompressed to a cToken account pending closure are burned without decreasing cmint supply

Severity: Low	Impact: Medium	Likelihood: Low
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/compress_and_close.rs programs/compressed-token/program/src/compressed_token/transfer2/processor.rs	Status: Fixed	

Description:

In CompressAndClose operations, the cToken account is not immediately closed after executing the compression. Instead, the process happens in two phases:

1. `process_compress_and_close()` sets the cToken balance to zero via `ctoken.base.amount.set(0)`
2. All accounts are closed in batch at the end of the instruction via `close_for_compress_and_close()`

Between these two phases, if a Decompress operation targets the same cToken account within the same transaction, tokens will be added to the account. However, close_for_compress_and_close() will still execute the account closure by transferring all SOL out, resizing the account to zero, and reassigning ownership.

This results in the decompressed tokens being permanently burned. Critically, the cmint supply is not decreased to reflect this burn, creating a supply accounting mismatch where the recorded total supply exceeds the actual circulating tokens.

Recommendations:

Check that the balance is still zero before closing the CToken account.

Customer Response:

Fixed in PR #2234

Fix Review:

Looks good

L-03: Decompressing a compressed-only account might increase the delegation amount by more than it should

Severity: Low	Impact: Medium	Likelihood: Low
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/decompress.rs	Status: Acknowledged	

Description:

When decompressing a compressed-only account we increase the delegated amount on the destination address by the delegated amount in the compressed-only account.

However, the delegated amount in the compressed-only account might be greater than the amount held in the account (due to the owner spending some amount after the delegation), in that case we should only increase by the amount held in the compressed account as this is the amount that's effectively delegated.

Recommendations:

Cap the increase of the delegated amount by the amount held in the compressed-only account.

Customer Response:

Acknowledged. The current design is consistent with the SPL design, where a delegated amount can be greater than the account balance and applies to the new balance when it increases.

L-04: Permanent delegate can decompress a compressed-only account only into an account owned by the original owner

Severity: Low	Impact: Low	Likelihood: Medium
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/decompress.rs	Status: Acknowledged	

Description:

When decompressing a compressed-only account we require it to be decompressed into an account with the same owner (or in the case of ata also to the same address).That restriction is applied also when the permanent delegate signs the decompression.

This is contrary to the definition of the role of the permanent delegate, which should allow him to transfer from any account to any account without any restrictions.

Recommendations:

Lift this restriction when the decompression is initiated by the permanent delegate

Customer Response:

Acknowledged

L-05: Frozen CToken account can't be closed, unlike SPL token specs

Severity: Low	Impact: Low	Likelihood: Low
Files: programs/compressed-token/program/src/ctoken/close/processor.rs	Status: Fixed	

Description:

Frozen CToken accounts can't be closed, the CloseAccount would revert for such case.This doesn't follow SPL token specs which allow closing an account even under the frozen state.

Recommendations:

Modify the CloseAccount ix to allow closing a frozen account

Customer Response:

Fixed in PR #2243

Fix Review:

Looks good

L-06: Compression authority can add CompressedOnly extension to compressed token accounts that don't need it

Severity: Low	Impact: Low	Likelihood: Low
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/compress_and_close.rs	Status: Fixed	

Description:

While executing compress and close, we check that the CompressedOnly extension is present when necessary. However, we don't check that it's not present when it's not necessary.This would allow the compression authority to add a CompressedOnly extension to account that don't need them, unnecessarily restricting the users during decompression.

Recommendations:

Add a check to enforce that the CompressedOnly extension isn't present when not necessary.

Customer Response:

Fixed in PR #2238

Fix Review:

Looks good

L-07: `max_top_up` max value isn't sufficient to support many use cases

Severity: Low	Impact: Low	Likelihood: Low
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/mod.rs	Status: Fixed	

Description:

max_top_up is of u16 type, meaning the max value is 65,536.This might not be sufficient to support many average user cases. Assuming a max top up of 10K lamports, the required top up for an ix with 10 accounts might exceed the max value, forcing the caller to not specify the limit.

Recommendations:

Consider using a type with more bits, or use the existing type but increase the units it represents (e.g. units would be 1000 lamports, so when max_top_up=100 the max top up would be 100K lamports)

Customer Response:

Fixed in PR #2265

Fix Review:

Looks good

L-08: Compressed only delegation should apply the delegated amount if delegation address is the same

Severity: Low	Impact: Low	Likelihood: Medium
Files: programs/compressed-token/program/src/compressed_token/transfer2/compression/ctoken/decompress.rs	Status: Fixed	

Description:

During decompression of a 'compressed-only' account, if the delegate is already set in the destination CToken account, then we early return and don't apply the delegation.

This makes sense when the delegate address is different, but when it's the same we should increase the delegated amount accordingly.

Recommendations:

Increase the delegated amount when the delegate address in the compressed account is the same as the destination.

Customer Response:

Fixed in PR #2242

Fix Review:

Looks good

L-09: Permanent delegate can't burn

Severity: Low	Impact: Low	Likelihood: Medium
Files: programs/compressed-token/program/src/ctoken/burn.rs	Status: Acknowledged	

Description:

Per token 2022 specs the permanent delegate should be able to transfer or burn from any token account.However, the compressed-token program supports only transferring by the permanent delegate, burning by the permanent delegate isn't supported (the permanent delegate can still burn by transferring the funds into an account they own and burning from there, but burning directly isn't supported).

Recommendations:

Consider adding support for burning by the permanent delegate.

Customer Response:

Acknowledged

L-10: During mint decompression we shouldn't allow `rent_payment` to be 0

Severity: Low	Impact: Low	Likelihood: Medium
Files: programs/compressed-token/program/src/compressed_token/mint_action/actions/decompress_mint.rs	Status: Fixed	

Description:

decompressing mint it doesn't make sense to allow action.rent_payment to be 0

0 means immediate compression iirc, so it doesn't make sense to decompress just to immediately compress.

It can also be used by an attacker to keep moving it between the compressed and decompressed state in order to DoS using the mint account

Recommendations:

Don't allow the rent_payment to be 0, require it to be at least 2

Customer Response:

Fixed in PR #2267

Fix Review:

Looks good

L-11: An attacker can create an account before mint creation to avoid restricted extension detection

Severity: Low	Impact: Medium	Likelihood: Low
Files: programs/compressed-token/program/src/ctoken/create.rs	Status: Acknowledged	

Description:

When creating a token account, we read the mint to see if it has any restricted extensions.If it's not owned by the token22 program, then we assume there are no restricted extensions.This can allow an attacker to create a token account before the mint is created, and avoid detection of the restricted extensions.

Recommendations:

Check that the mint is a valid mint - it should be owned and initialized by one of the token programs, or the address should be a valid compressed mint address.

Customer Response:

Acknowledged

Informational Severity Issues

I-01: Compressed token accounts with zero amounts can be created for restricted mints

Files:

programs/compressed-token/anchor/src/process_transfer.rs

Description:

Anchor's transfer ix allows transferring between compressed accounts by specifying a list of inputs and outputs where the sum of inputs minus the sum of outputs per mint is zero.

This ix doesn't support specifying extensions, therefore we can never use a compressed account from a restricted mint (since they have the compressed-only extension).But we can create new output accounts without specifying any input accounts, since this would pass the zero-sum check.

This would allow for example having zero amount transfers when a mint is paused.

Customer Response:

Acknowledged

I-02: New rent exempt rate is covered by the fee payer but goes to the rent sponsor when closed

Files:

programs/compressed-token/program/src/compressed_token/mint_action/mint_output.rs

Description:

When topping up the decompressed account to match the new rent exemption rate, the difference is charged from the function caller, but when we close the account it goes to the rent sponsor. This doesn't seem fair towards the function caller.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.